

Sri Sivasubramaniya Siva College of Engineering, Chennai

(An Autonomous Institution Affiliated to Anna University)

Machine Learning Algorithms Laboratory

Experiment 4

Binary Classification using Linear and Kernel-Based Models

Name	MAHALINGAM S S
Register No.	3122247001034
Degree & Branch	M. Tech (Integrated) Computer Science & Engineering
Semester	V
Subject Code & Name	ICS1512 – Machine Learning Algorithms Laboratory
Academic Year	2026–2027 (Odd)
Batch	2024–2029

Aim

To classify emails as spam or ham using Logistic Regression and Support Vector Machine (SVM) classifiers and compare their performance using different evaluation metrics.

Objective

- Implement Logistic Regression for binary classification.
- Implement Support Vector Machine using different kernels.
- Tune Logistic Regression hyperparameters using GridSearchCV.
- Tune SVM hyperparameters using GridSearchCV.
- Compare classifiers using accuracy, precision, recall, F1 score and training time.
- Perform five-fold cross-validation.

Theory

1. Logistic Regression

Logistic Regression is a probabilistic algorithm for binary classification. It models the probability of class 1 using the sigmoid function:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}.$$

A threshold, normally 0.5, converts the probability into a class label. The model minimizes log-loss.

Regularization

Regularization controls model complexity and helps reduce overfitting.

- L1 regularization encourages sparse coefficients and can perform feature selection.
- L2 regularization penalizes large coefficients and generally improves generalization.
- C is the inverse regularization strength: smaller C means stronger regularization.

2. Support Vector Machine

SVM is a margin-based classifier that finds a separating hyperplane while maximizing the margin. The important training samples defining the boundary are support vectors.

- Linear kernel – for approximately linear separation.
- Polynomial kernel – captures polynomial relationships.
- RBF kernel – handles nonlinear decision boundaries.
- Sigmoid kernel – provides a sigmoid-based nonlinear transformation.

The parameter C controls the trade-off between margin width and classification errors, while γ controls the influence of individual points for nonlinear kernels.

3. GridSearchCV

GridSearchCV exhaustively evaluates specified hyperparameter combinations using cross-validation and selects the configuration with the best validation score.

Dataset Description

The Spambase dataset was used. It contains numerical features extracted from email messages and a binary target indicating spam or ham. The executed Assignment 4 report records 4601 samples, 57 input features and one class column, with no missing values.

Preprocessing

- Loaded the Spambase CSV file.
- Checked dataset shape, columns and missing values.
- Separated the 57 input features from the class label.
- Standardized the features using StandardScaler.
- Performed exploratory data analysis.
- Used an 80:20 train-test split with random state 42.

The training set contained 3680 samples and the test set contained 921 samples, with 57 features.

Exploratory Data Analysis

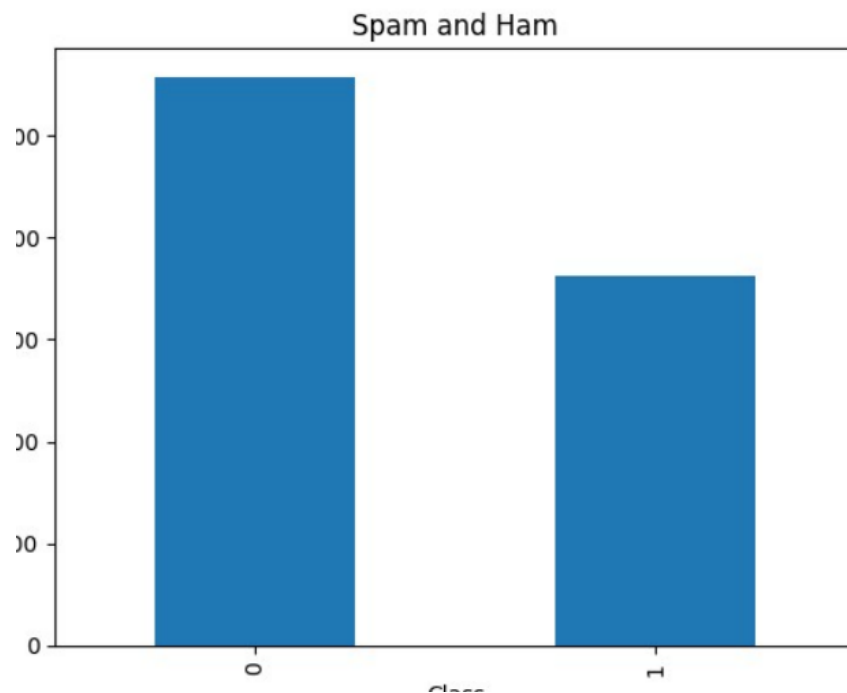


Figure 1: Spam and Ham Class Distribution

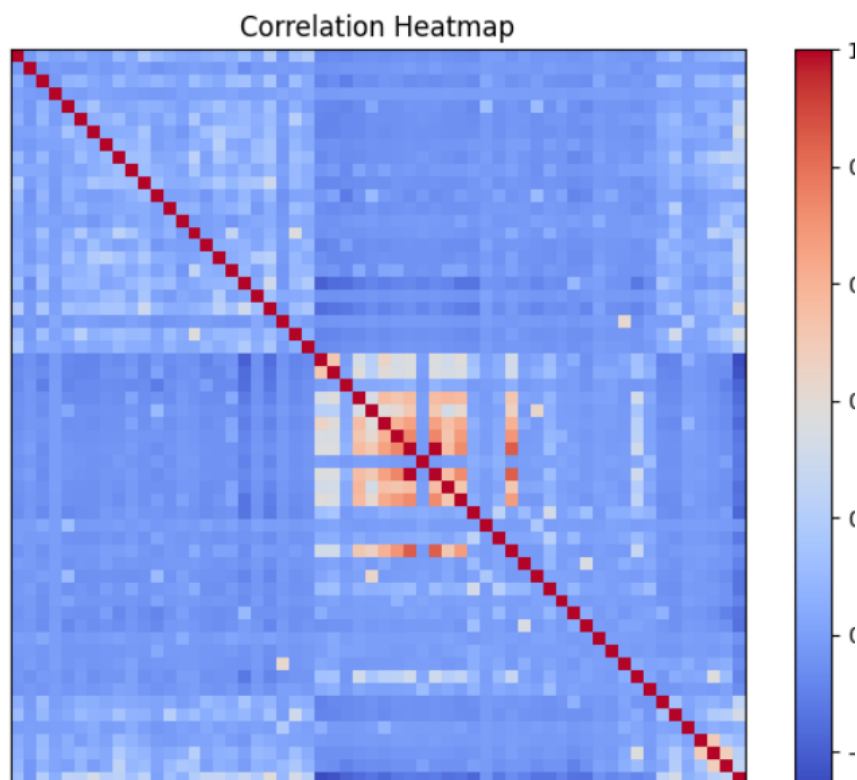


Figure 2: Correlation Heatmap

Implementation Details

The baseline Logistic Regression model used `max_iter=1000`. Logistic Regression was tuned using:

- $C \in \{0.1, 1, 10\}$
- `solver = liblinear`
- five-fold cross-validation

The best Logistic Regression configuration was $C = 10$, `solver liblinear`, with GridSearchCV accuracy 0.925815.

Four SVM kernels were trained: Linear, Polynomial (degree 3), RBF and Sigmoid.

The SVM GridSearchCV search space was:

- `kernel = Linear or RBF`
- $C \in \{1, 10\}$
- $\gamma = \text{scale}$
- five-fold cross-validation

The best SVM configuration was RBF with $C = 1$ and $\gamma = \text{scale}$, with GridSearchCV accuracy 0.931250.

Logistic Regression Performance

The saved executed experiment produced the following confusion matrix:

$$\begin{bmatrix} 506 & 25 \\ 49 & 341 \end{bmatrix}$$

The corresponding metrics are:

- Accuracy = 0.919653
- Precision = 0.931694
- Recall = 0.874359

- F1 Score = 0.902116

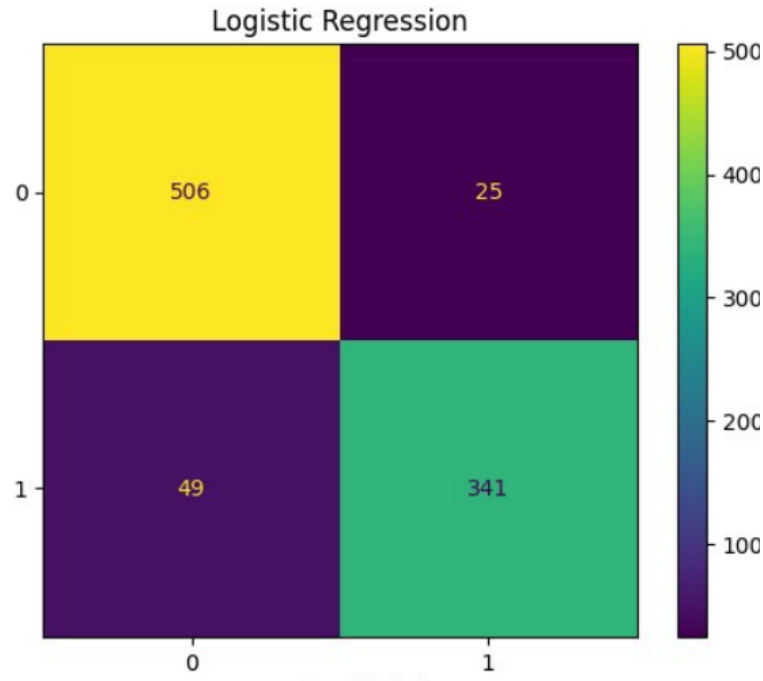


Figure 3: Confusion Matrix – Logistic Regression

SVM Kernel-wise Performance

The executed experiment records the following test performance:

Table 1: SVM Kernel-wise Performance		
Kernel	Accuracy	F1 Score
Linear	0.925081	0.909091
Polynomial (degree 3)	0.889251	0.866492
RBF	0.934853	0.920635
Sigmoid	0.764387	0.629060

The RBF kernel achieved the highest test accuracy among the four kernels.

Best SVM Performance

The GridSearchCV-selected best SVM used an RBF kernel with $C = 1$ and $\gamma = \text{scale}$.

Its confusion matrix was:

$$\begin{bmatrix} 513 & 18 \\ 42 & 348 \end{bmatrix}$$

The resulting metrics are:

- Accuracy = 0.934853
- Precision = 0.950820
- Recall = 0.892308
- F1 Score = 0.920635

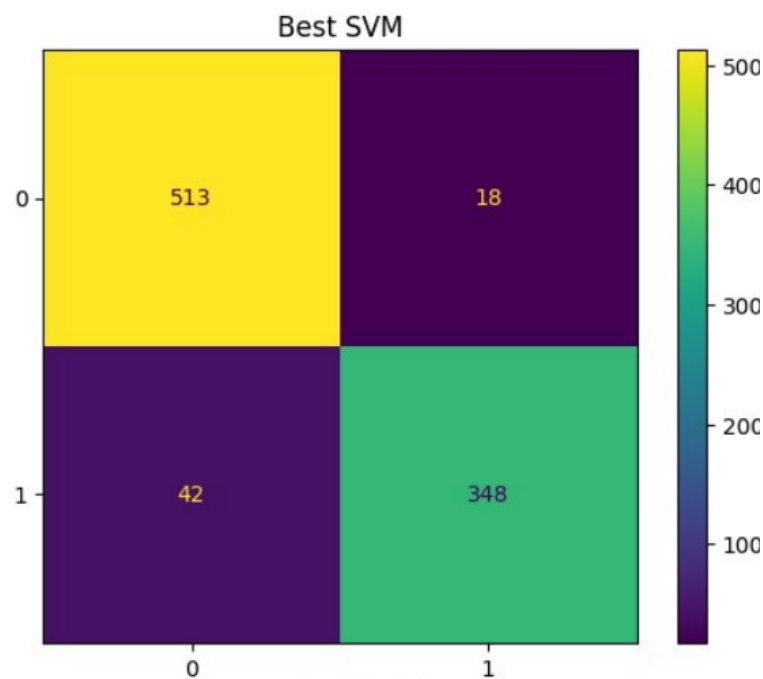


Figure 4: Confusion Matrix – Best SVM

Five-Fold Cross-Validation

Five-fold cross-validation was performed for Logistic Regression and the best SVM using the standardized dataset. The notebook constructs a table containing Fold 1 through Fold 5 and the average accuracy for both models.

The saved executed report preserves the cross-validation computation and summary statistics but not every individual dataframe value in text form. The recorded fold-accuracy ranges are:

- Logistic Regression: approximately 0.8402 to 0.9402.

- Best SVM: 0.8500 to 0.9500.

Hyperparameter Tuning Summary

Table 2: GridSearchCV Results		
Model	Best Parameters	Best CV Accuracy
Logistic Regression	$C = 10$, solver = liblinear	0.925815
SVM	$C = 1$, $\gamma = \text{scale}$, kernel = RBF	0.931250

Model Comparison

Table 3: Logistic Regression vs Best SVM		
Criterion	Logistic Regression	SVM
Accuracy	0.919653	0.934853
Model Complexity	Low	High
Training Time	Baseline measured	Minimum of tested SVM kernels
Interpretability	High	Low

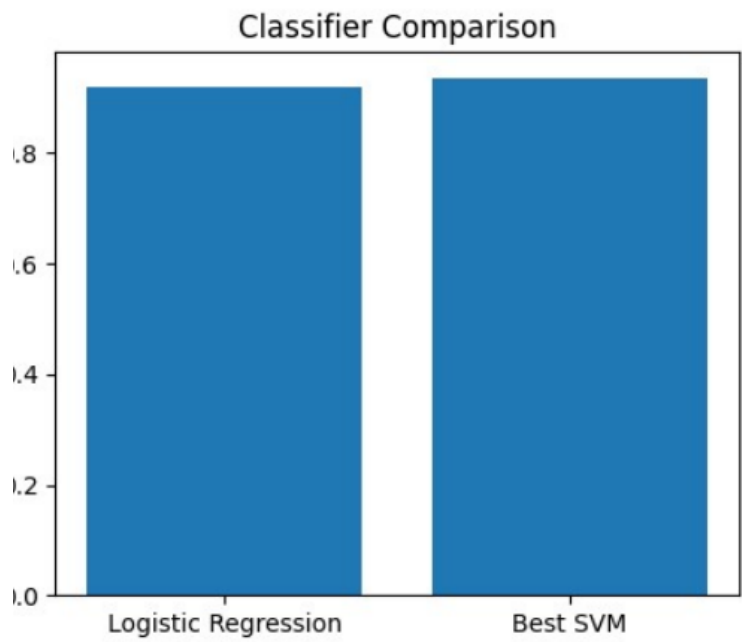


Figure 5: Classifier Accuracy Comparison

Observations

- The Spambase dataset contains 4601 email samples and 57 numerical input features.
- No missing values were found.
- Standardization was applied before model training.
- Logistic Regression achieved approximately 91.97% test accuracy.
- The best SVM achieved approximately 93.49% test accuracy.
- Among the tested SVM kernels, RBF performed best and Sigmoid performed worst.
- GridSearchCV helped select suitable hyperparameters using five-fold cross-validation.
- SVM achieved better test accuracy, while Logistic Regression remained simpler and more interpretable.

Observation Questions

1. Which classifier performed better?

The best SVM performed better on the test set, achieving approximately 93.49% accuracy compared with approximately 91.97% for Logistic Regression.

2. What is the effect of regularization in Logistic Regression?

Regularization penalizes large coefficients and controls model complexity. Stronger regularization can reduce overfitting, while weaker regularization permits a more flexible model.

3. How do SVM kernels affect performance?

Different kernels create different decision boundaries. The RBF kernel models nonlinear relationships effectively and produced the highest test accuracy in this experiment.

4. Comment on the bias–variance trade-off.

Simpler models tend to have lower variance and higher interpretability, while flexible nonlinear kernels can capture more complex patterns. Hyperparameter tuning helps balance flexibility and generalization.

Learning Outcomes

- Understood Logistic Regression for binary classification.

- Understood the working of Support Vector Machines.
- Implemented and compared different SVM kernels.
- Performed hyperparameter tuning using GridSearchCV.
- Evaluated models using accuracy, precision, recall and F1 score.
- Performed five-fold cross-validation.
- Compared classifiers based on predictive performance and model characteristics.

Conclusion

Logistic Regression and Support Vector Machine classifiers were implemented for spam/ham email classification using the Spambase dataset. Features were standardized and baseline as well as tuned models were evaluated. GridSearchCV selected $C = 10$ with the `liblinear` solver for Logistic Regression and an RBF SVM with $C = 1$ and $\gamma = \text{scale}$. The best SVM achieved higher test accuracy than Logistic Regression, showing the usefulness of a nonlinear decision boundary for this dataset. The experiment demonstrates the importance of preprocessing, kernel selection, hyperparameter tuning and cross-validation.

Complete Python Code

The following section contains **all code cells from the uploaded Assignment 4 notebook**, preserved in their original order.

```
% Code Cell 1
import time

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import cross_val_score

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import ConfusionMatrixDisplay

% Code Cell 2
from google.colab import files

uploaded = files.upload()

% Code Cell 3
data = pd.read_csv("spambase_csv.csv")

data.head()

% Code Cell 4
print("Shape :", data.shape)

print("\nColumns")
print(data.columns)

print("\nMissing Values")
print(data.isnull().sum())

data.info()
```

```

% Code Cell 5
X = data.drop("class", axis=1)

y = data["class"]

% Code Cell 6
scaler = StandardScaler()

X = scaler.fit_transform(X)

% Code Cell 7
data["class"].value_counts().plot(kind="bar")

plt.title("Spam and Ham")

plt.xlabel("Class")

plt.ylabel("Count")

plt.show()

% Code Cell 8
plt.figure(figsize=(8,6))

plt.imshow(data.corr(), cmap="coolwarm")

plt.colorbar()

plt.title("Correlation Heatmap")

plt.xticks([])

plt.yticks([])

plt.show()

% Code Cell 9
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training :", X_train.shape)

print("Testing :", X_test.shape)

```

```

% Code Cell 10
logistic = LogisticRegression(max_iter=1000)

start = time.time()

logistic.fit(X_train, y_train)

logistic_time = time.time() - start

logistic_pred = logistic.predict(X_test)

% Code Cell 11
accuracy = accuracy_score(y_test, logistic_pred)
precision = precision_score(y_test, logistic_pred)
recall = recall_score(y_test, logistic_pred)
f1 = f1_score(y_test, logistic_pred)

result = pd.DataFrame({
    "Metric": ["Accuracy", "Precision", "Recall", "F1 Score", "Training
Time"],
    "Value": [accuracy, precision, recall, f1, logistic_time]
})

result

% Code Cell 12
parameters = {
    "C": [0.1, 1, 10],
    "solver": ["liblinear"]
}

grid = GridSearchCV(
    LogisticRegression(max_iter=5000),
    parameters,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid.fit(X_train, y_train)

print("Best Parameters :", grid.best_params_)
print("Best Accuracy :", grid.best_score_)

% Code Cell 13
linear_model = SVC(kernel="linear")

```

```

start = time.time()

linear_model.fit(X_train, y_train)

linear_time = time.time() - start

linear_prediction = linear_model.predict(X_test)

% Code Cell 14
poly_model = SVC(
    kernel="poly",
    degree=3
)

start = time.time()

poly_model.fit(X_train, y_train)

poly_time = time.time() - start

poly_prediction = poly_model.predict(X_test)

% Code Cell 15
rbf_model = SVC(kernel="rbf")

start = time.time()

rbf_model.fit(X_train, y_train)

rbf_time = time.time() - start

rbf_prediction = rbf_model.predict(X_test)

% Code Cell 16
sigmoid_model = SVC(kernel="sigmoid")

start = time.time()

sigmoid_model.fit(X_train, y_train)

sigmoid_time = time.time() - start

sigmoid_prediction = sigmoid_model.predict(X_test)

% Code Cell 17
svm_parameters = {

```

```

        "kernel": ["linear", "rbf"],
        "C": [1, 10],
        "gamma": ["scale"]
    }

svm_grid = GridSearchCV(
    SVC(),
    svm_parameters,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

svm_grid.fit(X_train, y_train)

print("Best Parameters :", svm_grid.best_params_)
print("Best Accuracy :", svm_grid.best_score_)

% Code Cell 18
logistic_result = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1 Score",
        "Training Time"
    ],
    "Value": [
        accuracy_score(y_test, logistic_pred),
        precision_score(y_test, logistic_pred),
        recall_score(y_test, logistic_pred),
        f1_score(y_test, logistic_pred),
        logistic_time
    ]
})

logistic_result

% Code Cell 19
svm_result = pd.DataFrame({
    "Kernel": [
        "Linear",
        "Polynomial",
        "RBF",
        "Sigmoid"
    ],
    "Accuracy": [

```

```

        accuracy_score(y_test, linear_prediction),
        accuracy_score(y_test, poly_prediction),
        accuracy_score(y_test, rbf_prediction),
        accuracy_score(y_test, sigmoid_prediction)
    ],
    "F1 Score": [
        f1_score(y_test, linear_prediction),
        f1_score(y_test, poly_prediction),
        f1_score(y_test, rbf_prediction),
        f1_score(y_test, sigmoid_prediction)
    ],
    "Training Time": [
        linear_time,
        poly_time,
        rbf_time,
        sigmoid_time
    ]
})

svm_result

% Code Cell 20
ConfusionMatrixDisplay.from_predictions(
    y_test,
    logistic_pred
)

plt.title("Logistic Regression")

plt.show()

% Code Cell 21
best_model = svm_grid.best_estimator_

best_prediction = best_model.predict(X_test)

ConfusionMatrixDisplay.from_predictions(
    y_test,
    best_prediction
)

plt.title("Best SVM")

plt.show()

% Code Cell 22
logistic_cv = cross_val_score(

```

```

        logistic,
        X,
        Y,
        cv=5,
        scoring="accuracy"
    )

svm_cv = cross_val_score(
    best_model,
    X,
    Y,
    cv=5,
    scoring="accuracy"
)

cv_result = pd.DataFrame({
    "Fold": [
        "Fold 1",
        "Fold 2",
        "Fold 3",
        "Fold 4",
        "Fold 5",
        "Average"
    ],
    "Logistic Regression": [
        logistic_cv[0],
        logistic_cv[1],
        logistic_cv[2],
        logistic_cv[3],
        logistic_cv[4],
        logistic_cv.mean()
    ],
    "SVM": [
        svm_cv[0],
        svm_cv[1],
        svm_cv[2],
        svm_cv[3],
        svm_cv[4],
        svm_cv.mean()
    ]
})

cv_result

% Code Cell 23
tuning_result = pd.DataFrame({
    "Model": [

```

```

        "Logistic Regression",
        "Support Vector Machine"
    ],
    "Search Method": [
        "GridSearchCV",
        "GridSearchCV"
    ],
    "Best Parameters": [
        str(grid.best_params_),
        str(svm_grid.best_params_)
    ],
    "Best CV Accuracy": [
        grid.best_score_,
        svm_grid.best_score_
    ]
})

tuning_result

% Code Cell 24
comparison = pd.DataFrame({
    "Criterion": [
        "Accuracy",
        "Model Complexity",
        "Training Time",
        "Interpretability"
    ],
    "Logistic Regression": [
        accuracy_score(y_test, logistic_pred),
        "Low",
        logistic_time,
        "High"
    ],
    "SVM": [
        accuracy_score(y_test, best_prediction),
        "High",
        min(linear_time, poly_time, rbf_time, sigmoid_time),
        "Low"
    ]
})

comparison

% Code Cell 25
models = ["Logistic Regression", "Best SVM"]

scores = [

```

```
    accuracy_score(y_test, logistic_pred),  
    accuracy_score(y_test, best_prediction)  
]  
  
plt.figure(figsize=(5,4))  
  
plt.bar(models, scores)  
  
plt.ylabel("Accuracy")  
  
plt.title("Classifier Comparison")  
  
plt.show()  
  
% Code Cell 26
```

References

- [Scikit-learn Documentation – Logistic Regression.](#)
- [Scikit-learn Documentation – Support Vector Machines.](#)
- [Scikit-learn Documentation – GridSearchCV and Cross-Validation.](#)
- [Spambase Dataset – UCI Machine Learning Repository / Kaggle.](#)